

From Continuous Deployment with Jenkins Pipelines to MCP and LLM.



[marjune](#)

Follow

5 min read

.

Jun 26, 2025

With my progress in creating my own [MCP \(Model Context Protocol\)](#) server to manage a Kubernetes cluster, my primary goal is to learn how to utilize LLM for tasks beyond code generation and public knowledge questions. As I discussed in my previous article,

[*How I Built a Personal Docker-based MCP to manage a K8s cluster.*](#)

I created a few simple tools to interface with a Kubernetes cluster. Moving forward, I want to add more capability where, instead of using a CI/CD pipeline to deploy my application, LLM will take care of the deployment on my behalf, based on certain standards, so that LLM will not assume how to proceed. Since LLM can perfectly reason out, I'll communicate in a way that makes it seem like my intelligent assistant

is handling the task, removing the technical overhead associated with operating tools like Jenkins or any other CI/CD.

Application Deployment

I created a pipeline before where I used Jenkins as my CI/CD tool for deployment,

[Blue/Green Deployment in Kubernetes using Jenkins Declarative Pipeline.](#)

With the same application as Blue/Green. I'll add more tools to my *MCP*, like deployment from start to finish.

MCP Tools

<https://github.com/mrjbtc/mcp-k8s/tree/main>

To help the LLM understand how to deploy my application, I created a tool that outputs a clear, step-by-step description of the deployment process.

mcp-k8s/src/tools/deployment.py at main · mrjbtc/mcp-k8s

Contribute to mrjbtc/mcp-k8s development by creating an account on GitHub.
github.com

```
@mcp.tool(  
    annotations={  
        "title": "How to deploy blue green",  
        "readOnlyHint": True,  
        "destructiveHint": False,
```

```

        "openWorldHint": False
    }
)
async def how_to_deploy_blue_green():
    """ These are the steps on how to deploy Blue Green deployment """
    return """
        1. Query the existing blue deployment to get the number of replicas it's
        running. This ensures the green deployment will scale to match the current
        load.

        2. Update the svc-web-app-test service to point to the green deployment
        by modifying its selector to app=${GREEN-ENVIRONMENT} GREEN-ENVIRONMENT =
        web-app-a OR web-app-b. This will route traffic to the new version for pre-
        release validation.

        3. Update the image of the green deployment to the specified version/tag

        4. Scale the green deployment to match the replica count retrieved from
        the blue deployment (Step 1)

        5. Validate the green deployment by checking its health endpoint using
        the test service's IP=192.168.106.3 and NODE_PORT=30655 using the health
        check with the uri http://{IP}:{NODE_PORT} Monitor this endpoint for up to 3
        minutes. If it doesn't consistently return HTTP 200 OK, abort the deployment.

        6. Ask for approval to continue the deployment.

        7. Once the health check passes, pause and wait for manual approval to
        proceed with production switch.

        8. Update the production service svc-web-app to point to the green
        deployment by modifying its selector to app=${GREEN-ENVIRONMENT} GREEN-
        ENVIRONMENT = web-app-a OR web-app-b

        9. Promote the green deployment to "blue" by updating the labels from
        green to blue.

        10. Update the existing blue deployment's labels to indicate it is now
        the "green" (standby) environment.

        11. Finally, scale the old blue deployment down to zero replicas. Be
        careful to target the correct one (the previous production deployment, not
        the newly updated green).
    """

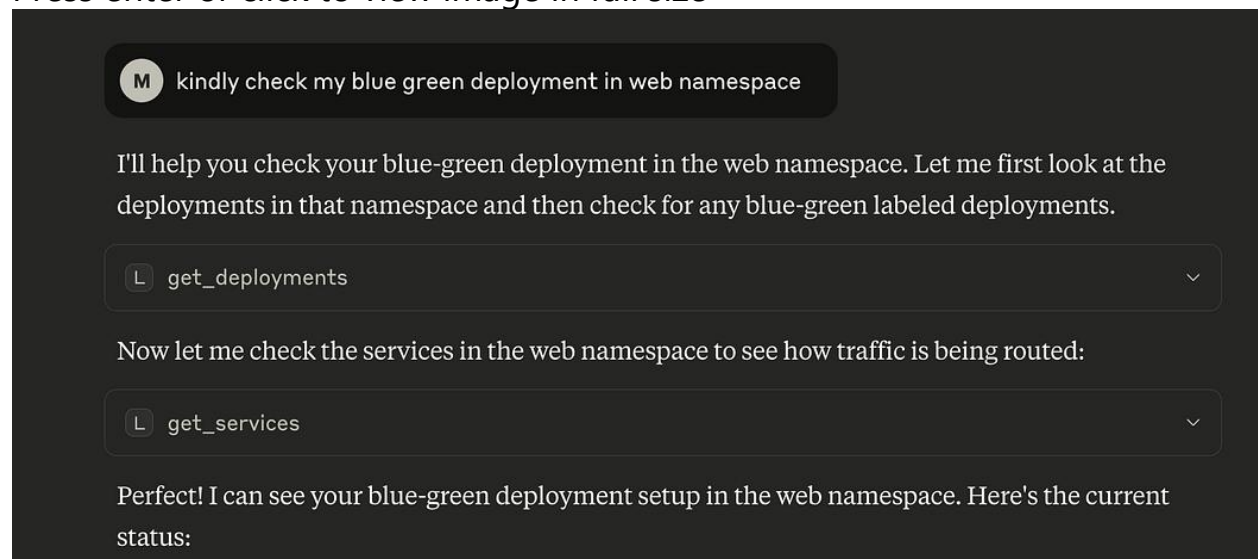
```

Other tools (<https://github.com/mrjbtc/mcp-k8s/tree/main/src/tools>) are available that support all these steps.

```
$ uv run python src/main.py
[mcp.run]
[available tools]:
get_deployments
describe_deployment
get_blue_green_deployment
set_image
scale_replicas
switch_blue_green_environment
how_to_deploy_blue_green
get_namespaces
get_pods
describe_pods
get_services
set_selector
```

Asking LLM to assist in deploying my application

As everything is prepared and ready to proceed with deployment. I started by asking,
Press enter or click to view image in full size



Based on my first interaction with LLM, I instructed it to check my Blue/Green deployment in the web namespace, and it decided to

invoke some of the tools available, which are `get_deployments`, and `get_services`.

Press enter or click to view image in full size

Blue-Green Deployment Status

Deployments:

1. web-app-a (Blue Environment)

- Status: **ACTIVE** ✓
- Environment label: `blue`
- Replicas: 2/2 running
- Image: `web-app:v1`
- Strategy: Recreate

2. web-app-b (Green Environment)

- Status: **INACTIVE** (Scaled to 0)
- Environment label: `green`
- Replicas: 0/0
- Image: `web-app:v1`
- Strategy: Recreate

Services:

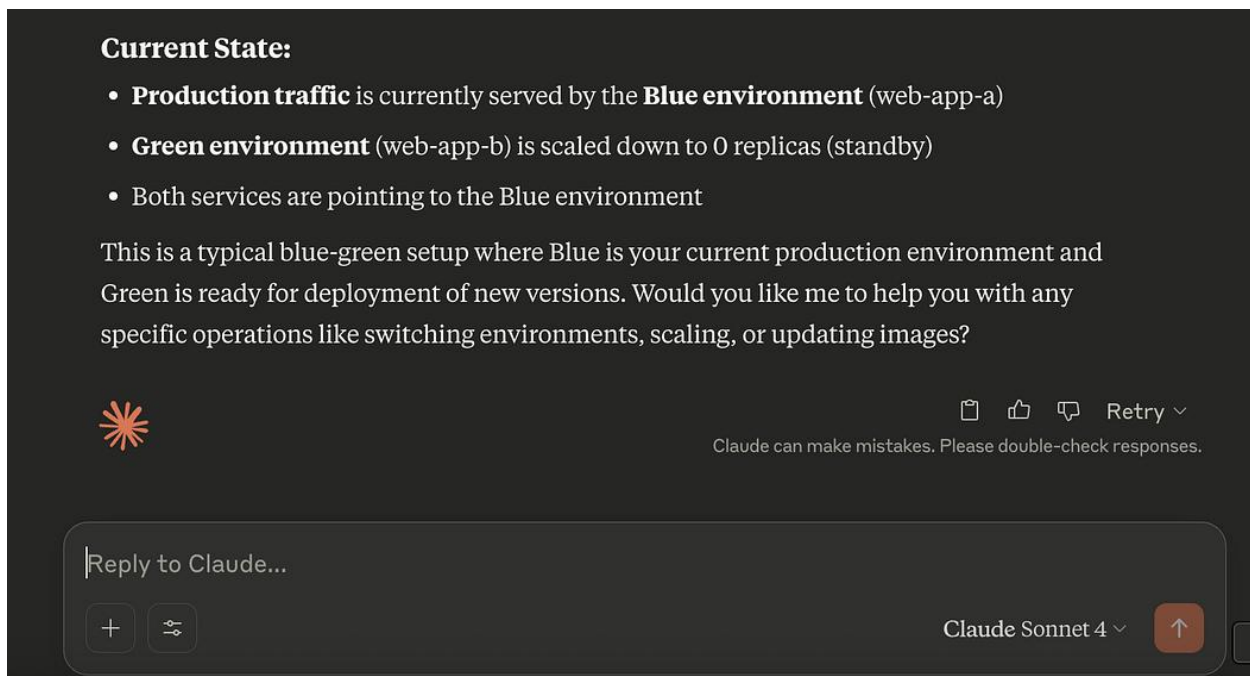
1. svc-web-app (Production service)

- Currently routing to: `web-app-a` (Blue)
- Port: 8081 → 80
- NodePort: 30654

2. svc-web-app-test (Test service)

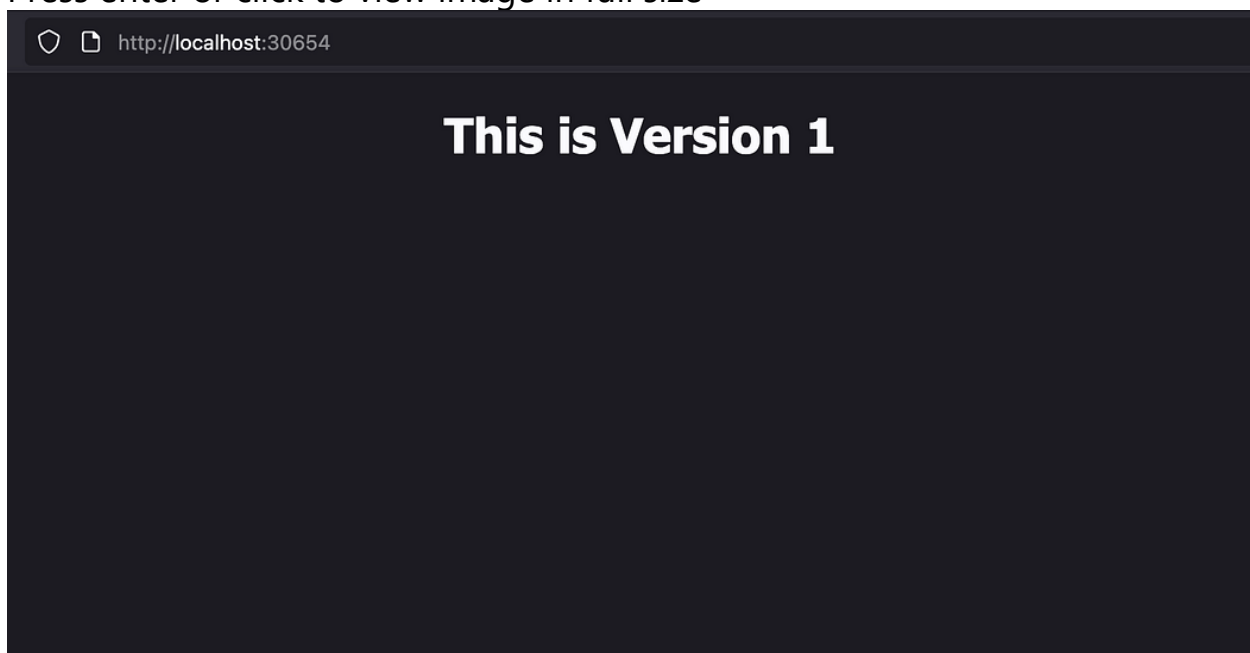
- Currently routing to: `web-app-a` (Blue)
- Port: 8081 → 80
- NodePort: 30655

Press enter or click to view image in full size



It summarizes the information from the tools. I check the production service, which says in the *NodePort:30654*

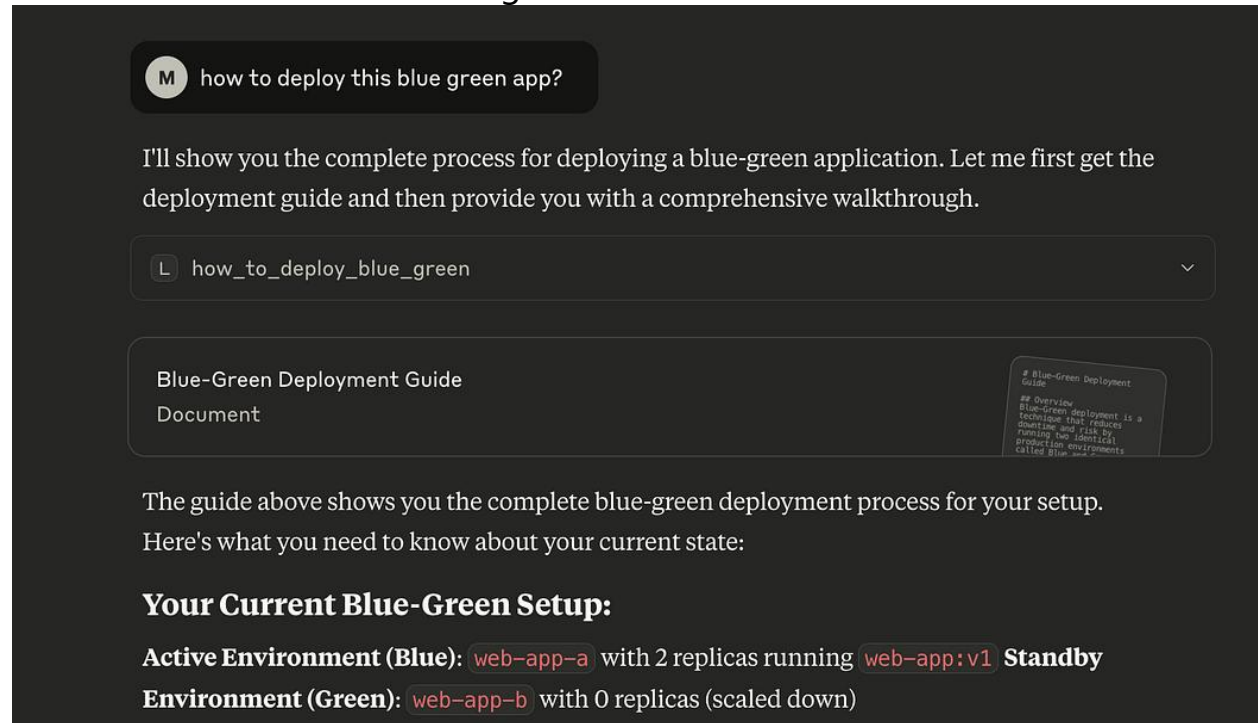
Press enter or click to view image in full size



And the production environment is serving *Version 1*.

I then follow up with the question on how to proceed with the blue/green deployment.

Press enter or click to view image in full size



Now, the `how_to_deploy_blue_green` tool is being invoked.

Press enter or click to view image in full size

Quick Deployment Example:

If you want to deploy a new version (let's say `web-app:v2`), here's what I can help you do:

1. Deploy to Green environment (safe testing):

- Update the green deployment image
- Scale it up to match blue
- Test via the test service

2. Switch production traffic (when ready):

- Point production service to green
- Swap the environment labels
- Scale down the old blue

Would you like me to help you deploy a specific version? If so, please let me know:

- What image version/tag you want to deploy
- Whether you want me to guide you through each step
- If you want to start with just the green environment setup for testing

I can execute each step for you using the kubectl commands, or would you prefer to see the commands to run them yourself?



📄 👍 💬 Retry ▾

Claude can make mistakes. Please double-check responses.

Reply to Claude...



Claude Sonnet 4 ▾



F

Then I gave a green signal to proceed.
Press enter or click to view image in full size

M please proceed to deploy web-app:v2 version

I'll help you deploy `web-app:v2` to your blue-green setup. Let me execute this step by step following the deployment process.

Step 1: Query Current Blue Deployment Replicas

L get_deployments

Let me get the current blue deployment details:

L describe_deployment

L describe_deployment

L get_deployments

Perfect! I can see that the current blue deployment (web-app-a) has **2 replicas**. Now let's proceed with the deployment.

Step 2: Point Test Service to Green Environment

L set_selector

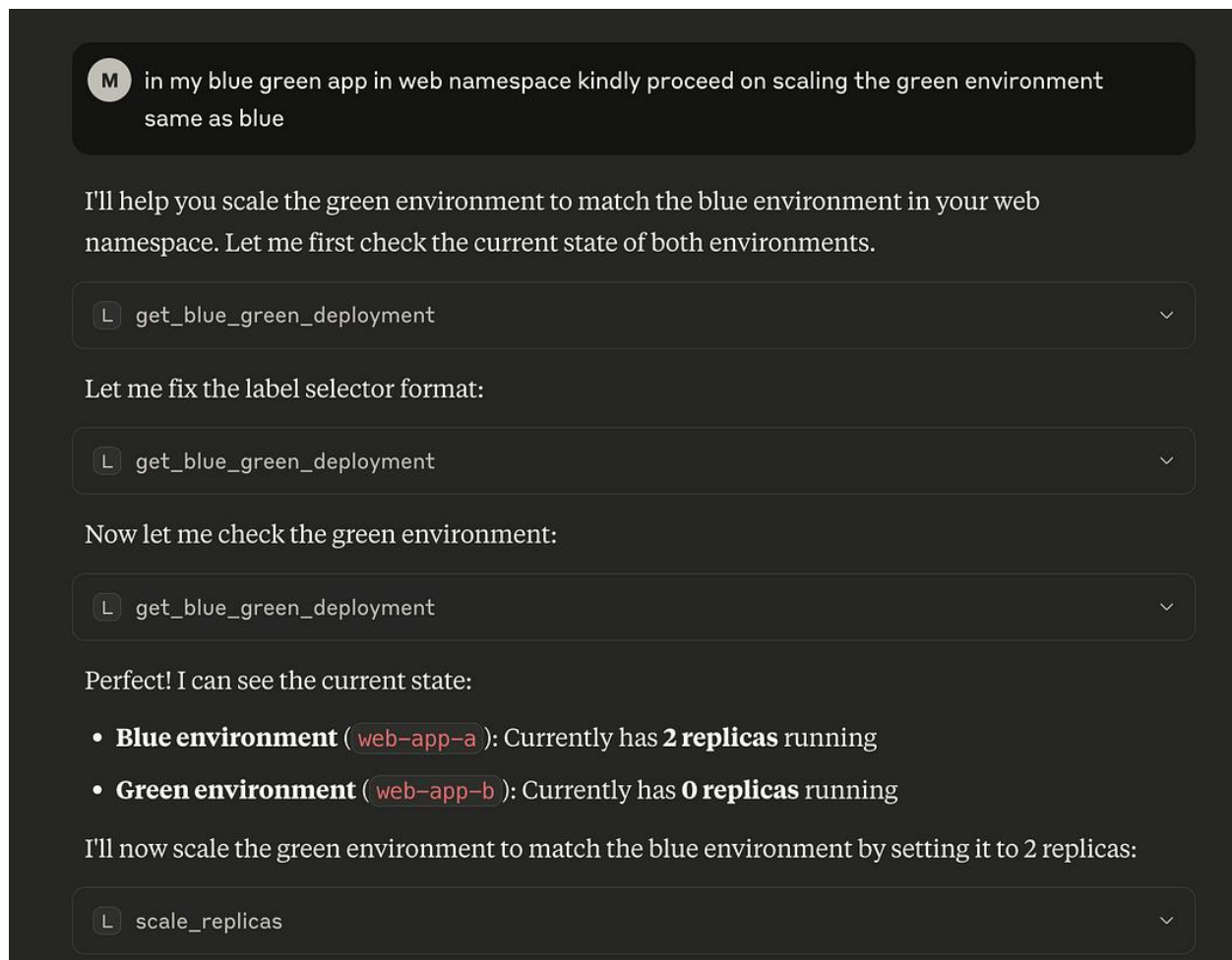
L set_selector

✅ **Step 2 Complete:** Test service now points to green environment (web-app-b)

Step 3: Update Green Deployment Image to v2

L set_image

Press enter or click to view image in full size



As the LLM proceeds, some tools are executed more than once due to some errors, that's why the model retried and re-construct the parameters.

Press enter or click to view image in full size

Let me verify the current state after scaling by checking the green deployment again:

L get_blue_green_deployment

Excellent! The scaling operation was successful. Let me also verify both environments are now properly aligned:

L get_blue_green_deployment

Perfect! The scaling operation has been completed successfully. Here's a summary of what was accomplished:

✓ Scaling Operation Completed

Blue Environment (`web-app-a`):

- Replicas: **2/2** (Ready)
- Image: `web-app:v1`
- Status: Available and Progressing

Green Environment (`web-app-b`):

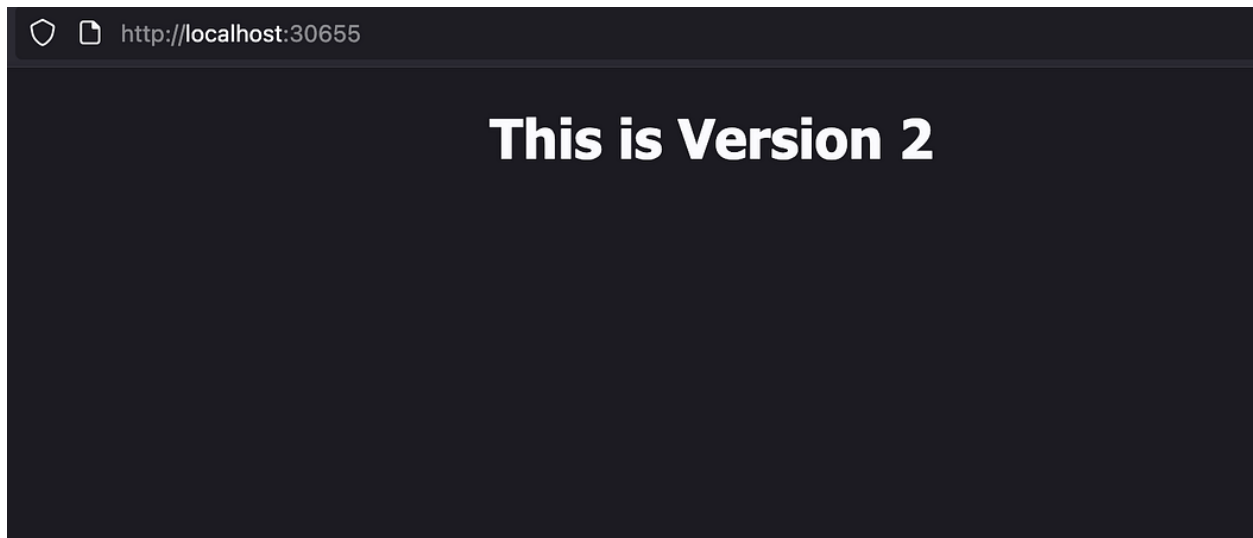
- Replicas: **2/2** (Ready) ✨ *Scaled up from 0*
- Image: `web-app:v2`
- Status: Available and Progressing

Both environments are now running with the same number of replicas (2 each) in the `web` namespace. The green environment has been successfully scaled up to match the blue environment and is ready for traffic or testing as needed.

You can now proceed with your blue-green deployment strategy, whether that's for testing the green environment before switching traffic or preparing for a seamless cutover.

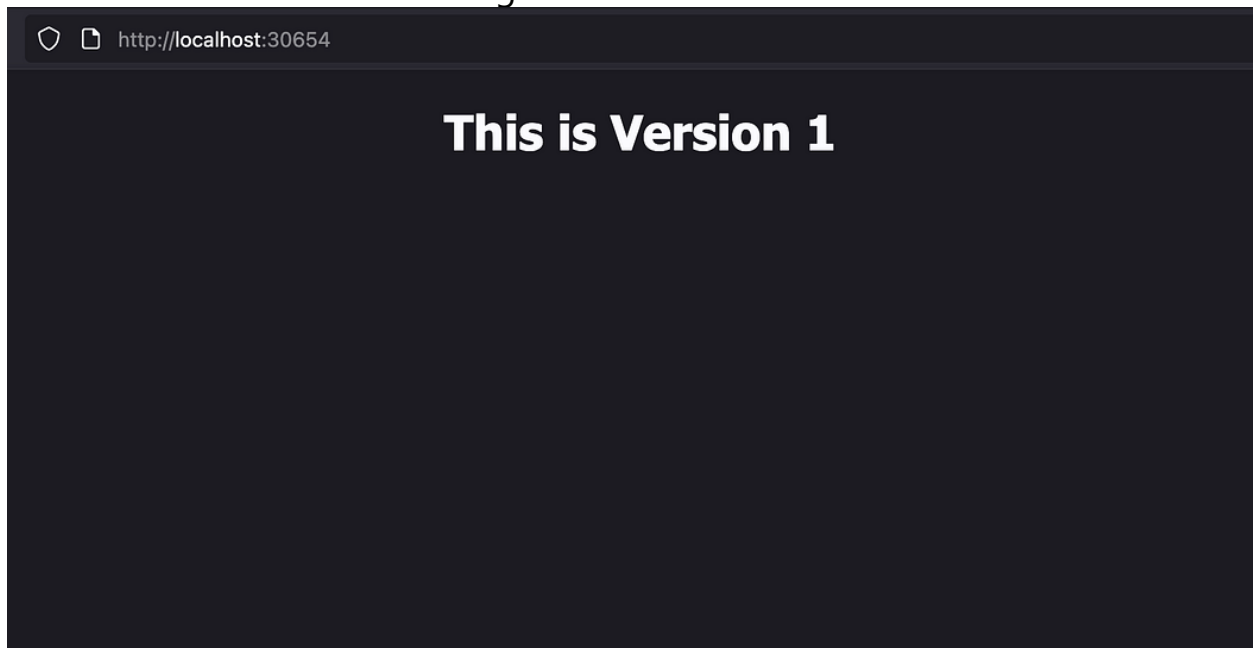
After confirming the steps above, I can now access my test service,
in *NodePort*: **30665**.

Press enter or click to view image in full size



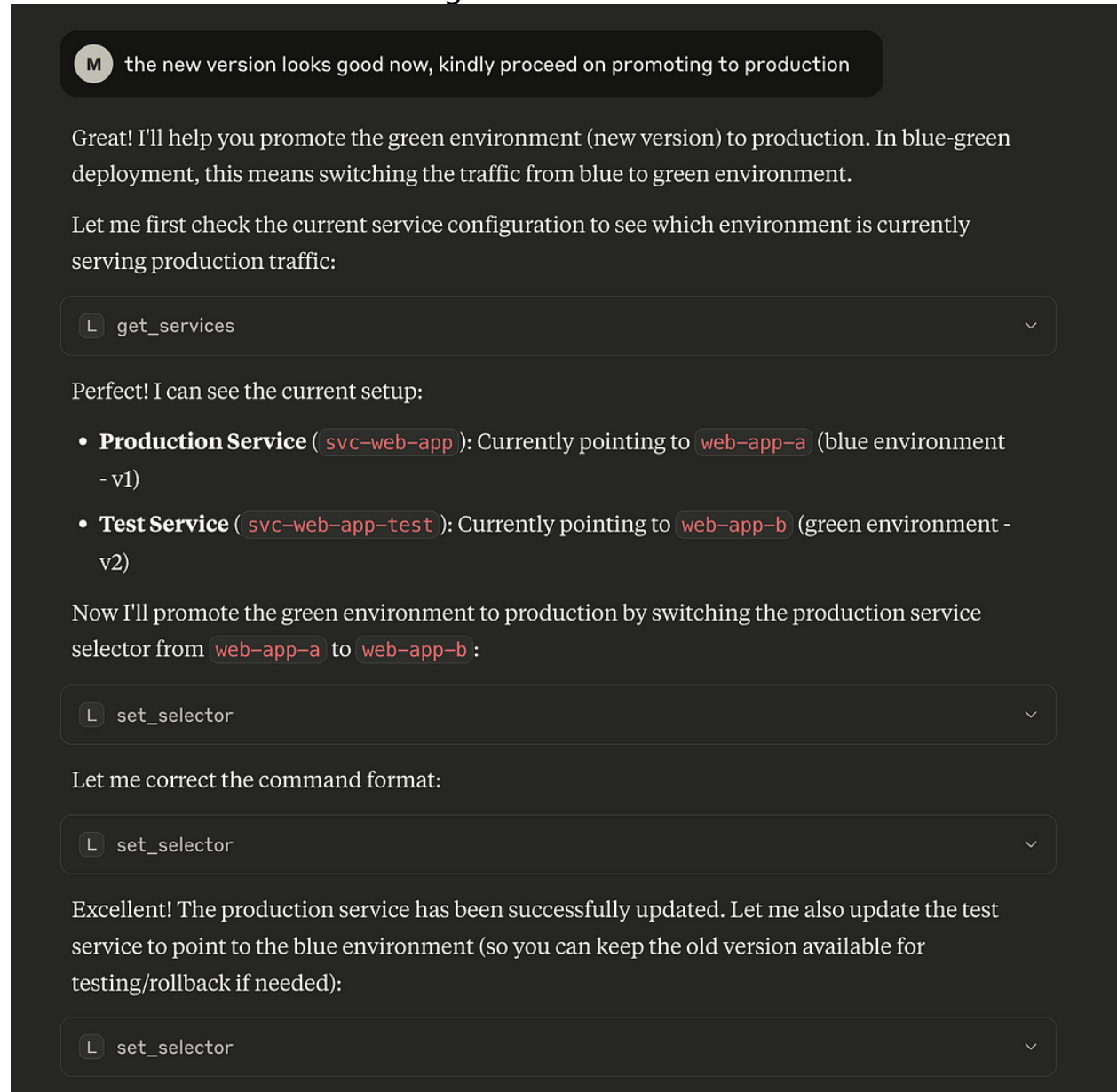
The test service endpoint is now pointing to green environment which is serving newly deployed *Version 2*.

And in the Blue environment still *Version 1* since I haven't instructed my assistant to proceed the green to production.
Press enter or click to view image in full size



After verifying that everything looks good. I tell my assistant to continue.

Press enter or click to view image in full size



Press enter or click to view image in full size

Perfect! Let me verify the final configuration by checking the services again:

L get_services

🎉 Production Promotion Complete!

The blue-green deployment promotion has been successfully completed. Here's the final configuration:

Production Environment (Now Live)

- **Service:** `svc-web-app` (NodePort: 30654)
- **Target:** `web-app-b` (Green environment)
- **Image:** `web-app:v2` 🌟 *New version now in production*
- **Replicas:** 2/2 (Ready)

Staging/Rollback Environment

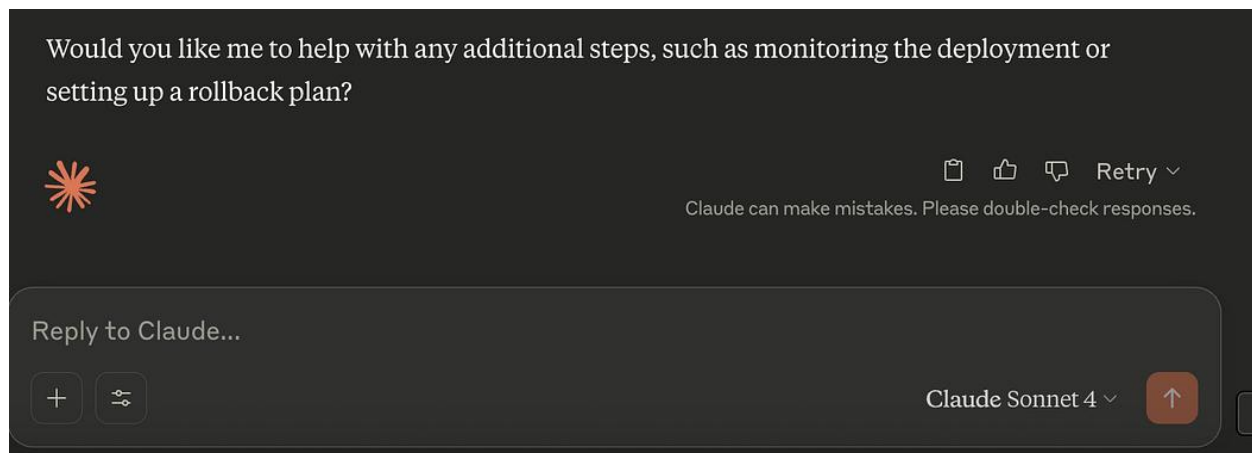
- **Service:** `svc-web-app-test` (NodePort: 30655)
- **Target:** `web-app-a` (Blue environment)
- **Image:** `web-app:v1` *Previous version kept for rollback*
- **Replicas:** 2/2 (Ready)

What Happened

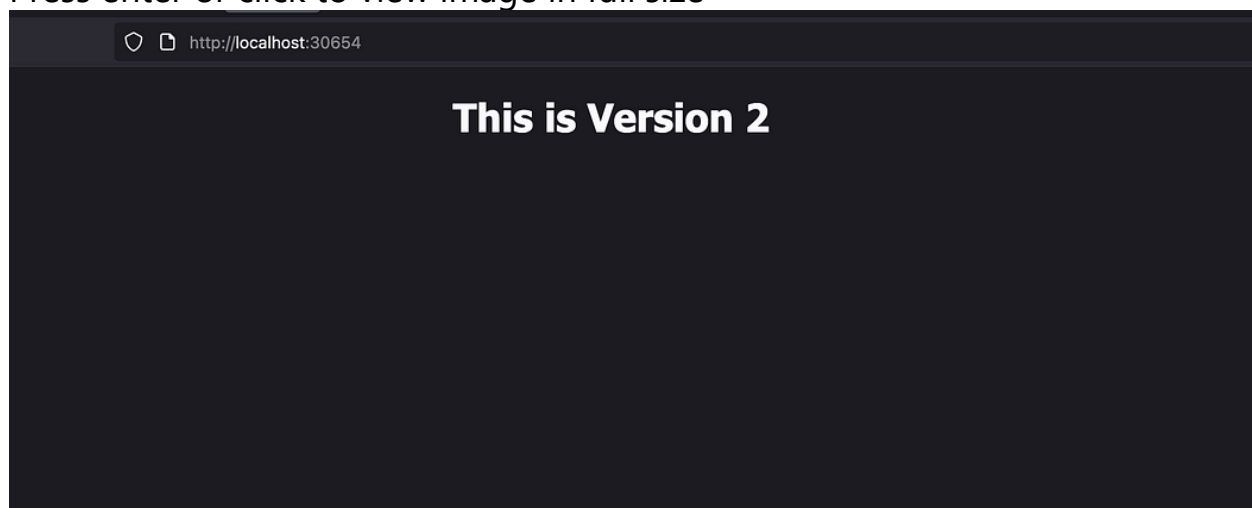
1. ✅ **Traffic Switched:** Production traffic now flows to the green environment (v2)
2. ✅ **Zero Downtime:** Seamless cutover with no service interruption
3. ✅ **Rollback Ready:** Previous version (blue) is still running and accessible via test service
4. ✅ **Both Environments Active:** Ready for instant rollback if needed

Your new version (`web-app:v2`) is now live in production! Users accessing your application will now see the updated version. The previous version remains available for testing or quick rollback if any issues arise.

Press enter or click to view image in full size



And there it is, deployment was successful. The production endpoint in **30654** port is now pointing to the new version of the application. Press enter or click to view image in full size



Final Thoughts

Deployment becomes effortless through a simple conversation.

If something goes wrong, I can instruct the LLM to initiate a rollback. Additionally, we can integrate more automated tests and allow the LLM to monitor system behavior post-deployment. Based on

predefined criteria, it can autonomously decide whether a rollback is necessary.

As we continue developing MCP, a good rule of thumb is to design tools that are focused and narrowly scoped. Avoid relying on the LLM to generate or execute high-risk, destructive commands. Always consider RBAC(*Role-Based Access Control*).